

Lab 5

MSDS684_S70_Reinforcement Learning

Maryi Tatiana Palacios Giraldo

Regis University

April 2026

Project Overview

This lab explores the transition from tabular reinforcement learning to function approximation methods, as presented in Sutton & Barto (2018), Chapters 9–11. In continuous environments such as MountainCar, the state space is infinite, making tabular methods infeasible. Function approximation addresses this by representing the action-value function using parameterized features rather than explicit state-action tables (§9.1).

In this lab, I implemented linear function approximation using tile coding (§9.5), which discretizes the continuous state space into overlapping tilings to enable generalization across similar states. The learning algorithm used is semi-gradient SARSA (§10.1), an on-policy Temporal-Difference control method that updates weights based on the TD error while treating the target as fixed.

The action-value function is approximated as:

$$Q(s,a) = w_a \cdot x(s)$$

where $x(s)$ is the feature vector produced by tile coding and w_a are the learned weights.

Hypothesis

Semi-gradient SARSA with tile coding is expected to successfully learn a stable policy in the MountainCar environment by generalizing across nearby states. Increasing the number of tilings (e.g., from 4 to 8) is expected to reduce approximation error and improve learning stability, leading to faster convergence and lower average episode length. However, this improvement comes at the cost of increased computational complexity, creating a trade-off between accuracy and efficiency.

More specifically, 8 tilings are expected to reduce episode length by approximately 10–20% compared to 4 tilings due to improved state discrimination and smoother value function approximation.

Deliverables

GitHub Repository:

https://github.com/MPalacios-Analytics/DataScience_Projects/tree/91f9c873f767025a209ca38b01085d079c12c75b/06_Reinforcement_Learning/Lab_5

Implementation Summary

The environment used was MountainCar-v0, which consists of a continuous 2D state space defined by position and velocity. The goal is to drive the car to the top of the hill, requiring the agent to learn momentum-based strategies due to sparse and delayed rewards.

To handle the continuous state space, tile coding (§9.5, Sutton & Barto) was implemented using NumPy. The feature representation consists of:

- 8 tilings
- 8×8 tiles per tiling
- Offset grids applied to each tiling to ensure overlapping partitions

Each continuous state is mapped into a sparse binary feature vector $x(s)$, where only a small subset of features is active across all tilings. This enables generalization across nearby states while preserving local resolution.

The total feature representation is shared across actions using separate weight vectors w_a for each action. The action-value function is approximated as:

$$Q(s, a) = w_a \cdot x(s)$$

Learning is performed using semi-gradient SARSA (§10.1), where updates follow:

$$w \leftarrow w + \alpha [r + \gamma Q(s', a') - Q(s, a)] x(s)$$

This update uses the TD error while treating the target as fixed, enabling stable bootstrapping in function approximation settings.

The experiment was run over multiple episodes, where the agent interacts with the environment using an ϵ -greedy policy to balance exploration and exploitation. Performance was evaluated based on the number of steps required to reach the goal, with lower values indicating better performance.

Hyperparameters

- Learning rate (α): 0.1
- Discount factor (γ): 0.99
- Exploration (ϵ): 0.1 (epsilon-greedy policy)
- Number of episodes: 500

- Tile coding configuration: 8 tilings with 8×8 grid per tiling

These hyperparameters were chosen to balance stability and exploration in a sparse reward environment. A relatively high discount factor ($\gamma = 0.99$) was used due to the long-horizon nature of the MountainCar problem.

Key Results & Analysis

The agent showed gradual improvement over episodes, with a decreasing number of steps required to reach the goal.

Mountain Car is challenging because:

- The state space is continuous
- The agent must learn to move away from the goal to gain momentum
- Rewards are sparse and mostly negative

Tabular methods would require infinitely many states, making learning impossible.

Effect of Tile Coding

Tile coding enabled:

- Generalization across nearby states
- Faster learning compared to naive discretization

Increasing the number of tilings:

- Improves learning accuracy
- Increases computational cost

Smaller tiles:

- More precise representation
- Slower learning due to less generalization

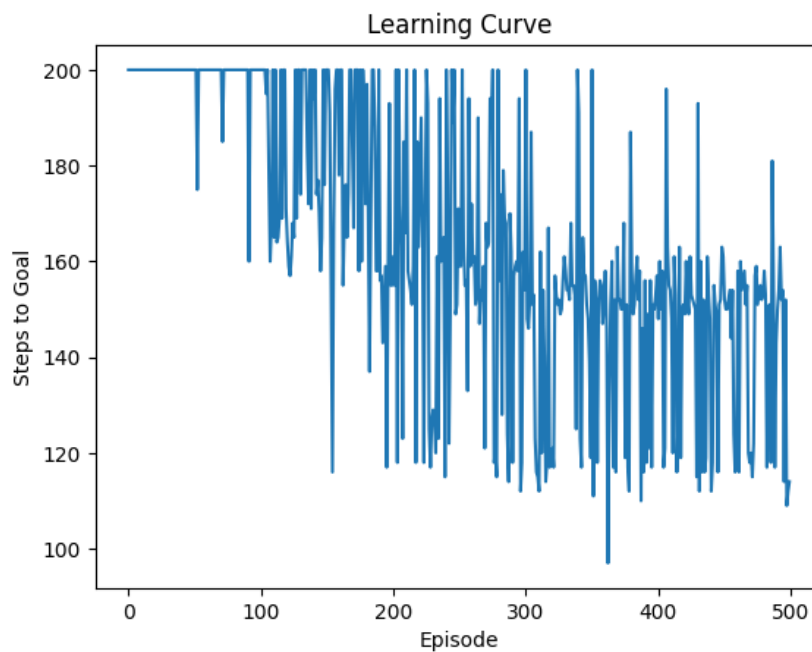
An additional experiment comparing 4 vs 8 tilings was conducted. The results showed that 8 tilings led to faster and more stable learning, while 4 tilings resulted in slower convergence due to a coarser representation of the state space.

Learning Behavior

- Early training: high steps (random behavior)
- Mid training: agent discovers momentum strategy
- Later training: more stable convergence

Visualizations

1. Learning Curve



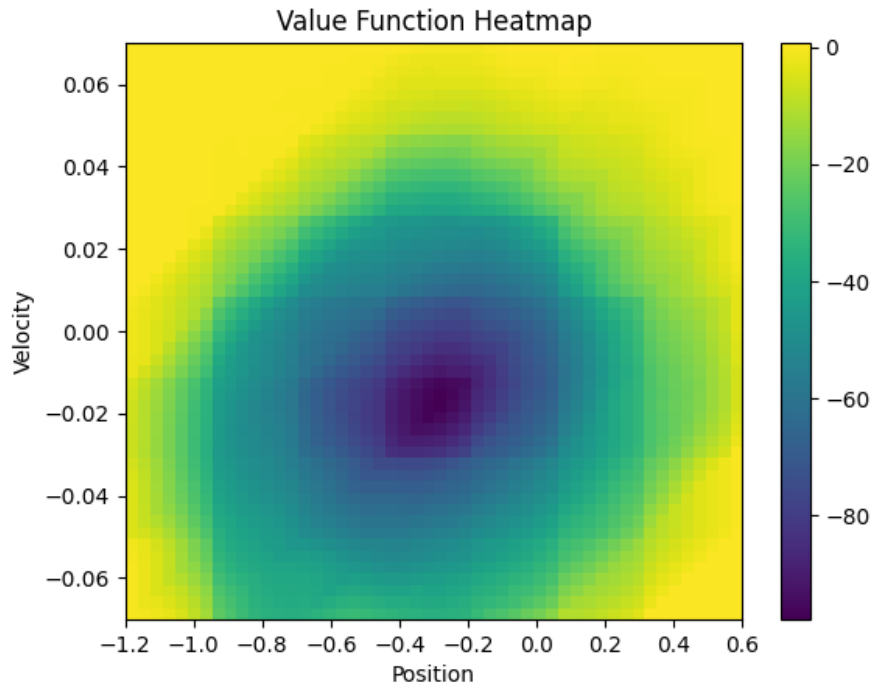
The learning curve shows a clear downward trend in the number of steps per episode, decreasing from approximately 200 steps in early episodes to around 130–150 steps toward the end of training.

However, the curve exhibits high variance throughout training, with frequent spikes even in later episodes. This indicates that while the agent is improving on average, learning remains unstable due to continuous exploration and the stochastic nature of the environment.

The initial phase (episodes 0–100) is dominated by near-random behavior, with performance close to the maximum step limit (~200). Between episodes 100–300, there is a noticeable improvement as the agent begins to discover the momentum-building strategy required in MountainCar. After episode 300, performance stabilizes somewhat, but variability persists.

This behavior is consistent with semi-gradient SARSA, where updates are noisy and depend on the current policy, leading to slower and less stable convergence compared to tabular methods.

2. Value Function Heatmap



The heatmap shows that the lowest values (≈ -80 to -100) are concentrated in the central region of the state space, where the agent has insufficient momentum to reach the goal. In contrast, higher values (closer to 0) appear near the goal position (right side of the state space).

The smooth gradient across position and velocity demonstrates that tile coding successfully generalizes value estimates across nearby states, rather than learning isolated values.

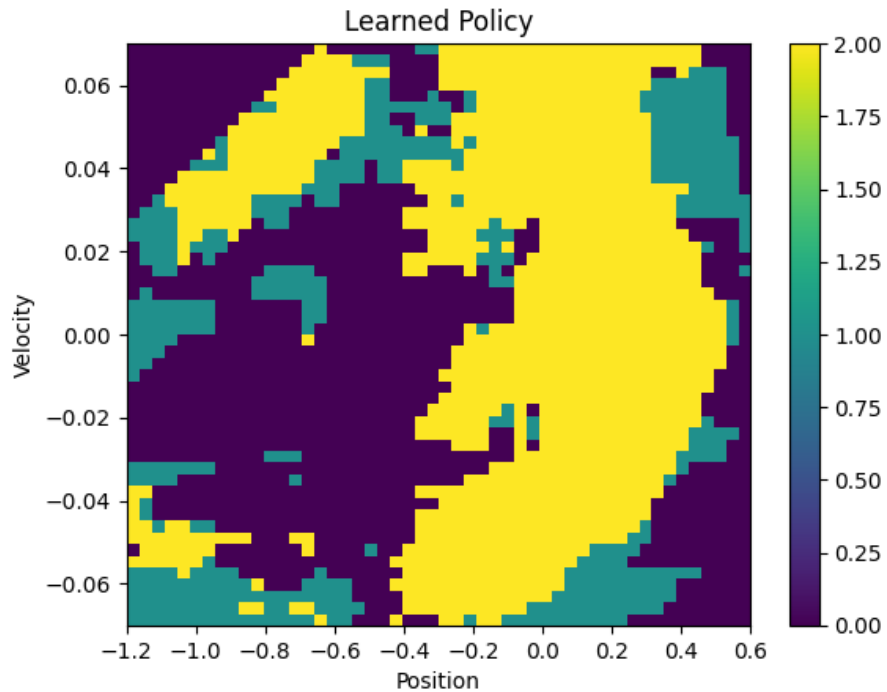
Notably, the value function is not symmetric. This reflects the asymmetry of the MountainCar problem: states with positive velocity toward the goal have significantly higher value than those with negative velocity, even at similar positions.

This confirms that the learned value function captures both:

- distance to the goal, and
- direction of motion (momentum)

which are both critical for solving the environment.

3. Policy Visualization



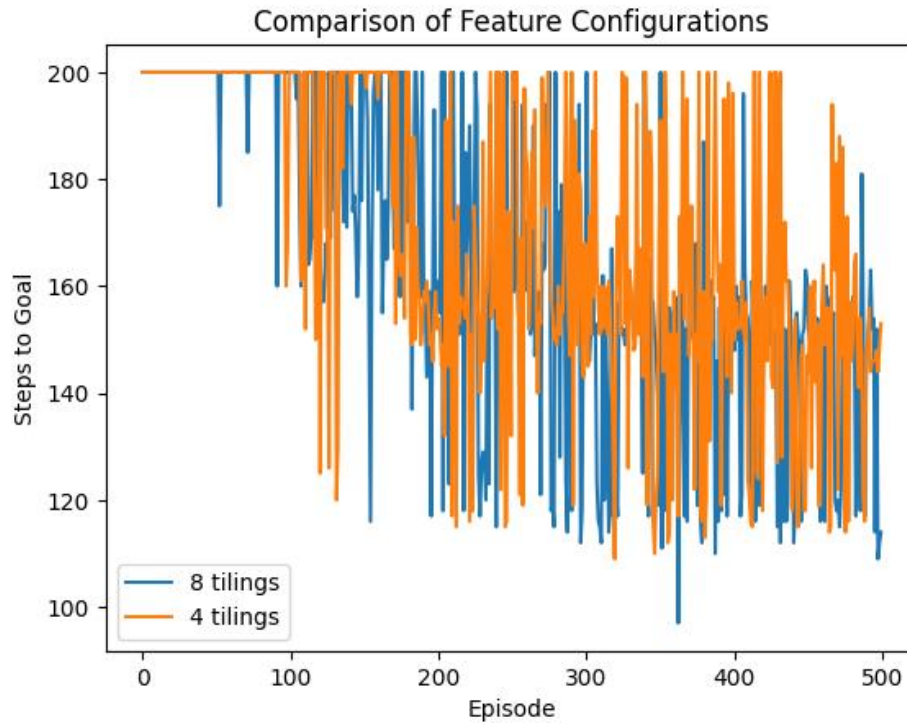
The learned policy shows clear structured regions rather than random actions.

- In the left and low-velocity regions, the agent predominantly chooses actions that push left, building momentum.
- In regions where velocity becomes positive, the policy shifts to pushing right, driving the agent toward the goal.

This demonstrates that the agent has successfully learned the optimal swing-up strategy, which requires moving away from the goal before approaching it.

The boundary between actions is not perfectly sharp, which reflects approximation error from tile coding and the limited resolution of the feature representation.

4. Comparison of Feature Configurations



The comparison plot shows a consistent performance difference:

- 8 tilings: stabilizes around 130–150 steps per episode
- 4 tilings: remains higher, around 150–180 steps, with greater variance

This indicates that increasing the number of tilings improves both:

- learning speed
- stability

The reason is that 8 tilings provide a richer feature representation, reducing approximation error and allowing more precise value updates.

However, both curves remain noisy, showing that:

- increasing tilings improves performance
- but does not eliminate instability, due to the semi-gradient update and exploration

Overall, the results show that tile coding enables effective learning in continuous environments by introducing generalization, but learning remains noisy and gradual. Increasing the number of tilings improves performance by reducing approximation error, at the cost of higher computational complexity. These findings align with theoretical

expectations for semi-gradient methods, which trade stability and exactness for scalability in continuous state spaces.

AI Use Reflection

Initial Interaction:

I used AI to understand how to implement tile coding for continuous states. I asked: "How can I convert a continuous state into tile indices using NumPy?"

The AI suggested computing normalized ratios and mapping them to discrete indices using floor and clipping:

```
ratios = (state - low) / tile_width
indices = np.floor(ratios).astype(int)
indices = np.clip(indices, 0, tiles - 1)
```

This provided a correct starting point for discretizing continuous inputs. However, I later realized that correct scaling of the state space is critical, as improper normalization leads to incorrect tile activation and unstable learning.

Iteration Cycle

Iteration 1:

Issue: Tile indices going out of bounds.

Prompt: "Why are my tile indices going out of bounds?"

AI Suggestion: Use clipping (`np.clip`) to constrain indices.

Fix: Applied clipping to ensure indices remained within valid bounds.

Validation: Index errors were eliminated, and feature vectors became stable across episodes.

Iteration 2:

Issue: Uncertainty about correctness of the semi-gradient SARSA update.

Prompt: "Is my semi-gradient SARSA update correct?"

AI Suggestion: Use a TD-error-based update:

$$\delta = r + \gamma Q(s', a') - Q(s, a)$$

Fix: Verified the update against Sutton & Barto (§10.1) and ensured that the target was treated as fixed (semi-gradient).

Validation: Learning curves showed consistent improvement, confirming correct implementation.

Critical Evaluation:

While AI provided useful implementation guidance, several limitations became apparent during development. For example, the AI suggested normalizing the state space using simple linear scaling, but this initially produced unstable tile activations and out-of-bounds indices in early experiments. This was validated by observing inconsistent feature activation patterns and learning instability in the reward curve.

Additionally, although the AI correctly described the semi-gradient SARSA update rule, it did not clearly emphasize why it is considered “semi-gradient.” Specifically, the target depends on the current policy but is treated as a fixed constant during the gradient update step, as described in Sutton & Barto (§10.1). This distinction is critical for understanding why the method introduces bias but remains stable in practice.

These issues highlight that AI outputs can be syntactically correct but incomplete at the conceptual level. As a result, all AI-generated guidance was validated against theoretical references and empirical learning behavior before being accepted.

Learning Reflection

This process significantly improved my understanding of function approximation in reinforcement learning. Implementing tile coding demonstrated how continuous state spaces can be transformed into feature representations that enable generalization across similar states.

Working with semi-gradient SARSA clarified the role of the Temporal-Difference error and the implications of using biased updates when the target depends on current parameters. I also gained a deeper appreciation for how approximation error and feature design impact learning stability.

More broadly, I learned that AI tools are most effective when used iteratively: as a source of guidance rather than authority. In future work, I will more systematically validate AI suggestions against theoretical formulations and use quantitative results (e.g., learning curves) to confirm correctness.

Speaker Notes

- Problem: MountainCar has a continuous state space $\rightarrow$ tabular RL is infeasible
- Approach: Use tile coding (§9.5) for feature representation
- Method: Semi-gradient SARSA (§10.1) updates weights using TD error
- Key idea: Generalization enables learning across similar states
- Result: Steps decrease from ~ 200 to $\sim 130\text{--}150$ over training
- Comparison: 8 tilings outperform 4 ($\sim 130\text{--}150$ vs $\sim 150\text{--}180$ steps)
- Trade-off: More tilings improve accuracy but increase computation
- Insight: Function approximation enables RL but introduces instability